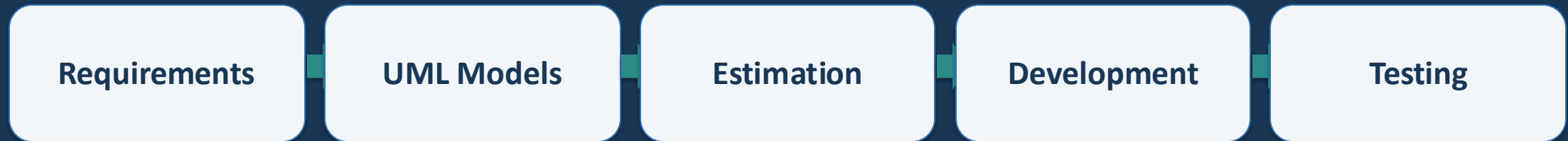


Unit – 2

Requirement Engineering & Object-Oriented Software Estimation



MCA – 212 • Object-Oriented Software Engineering

Dr Madhavi Dave

Learning Objectives

What students should be able to do after this module

01 • Requirements

Requirement Analysis, stakeholders, functional and non-functional requirements.

02 • Elicitation

Select interviews, workshops, observation, prototypes and scenarios.

03 • SRS

Write measurable, traceable, testable requirements.

04 • Change

Analyze impact and control evolving requirements.

05 • Estimation

Estimate OO size, effort, schedule and cost.

Connect requirements → UML → estimation → testing.

Apply concepts to case studies.

Why Requirements Engineering?

The cost of ambiguity grows as the project progresses

Requirements Engineering Lifecycle



What is a Requirement?

A requirement describes a needed service plus constraints.

Definition

A requirement specifies what the software must provide and the conditions or constraints under which it must operate.

Example • Examination System

“The system shall allow an authenticated student to start an examination only during the scheduled examination period.”

Function

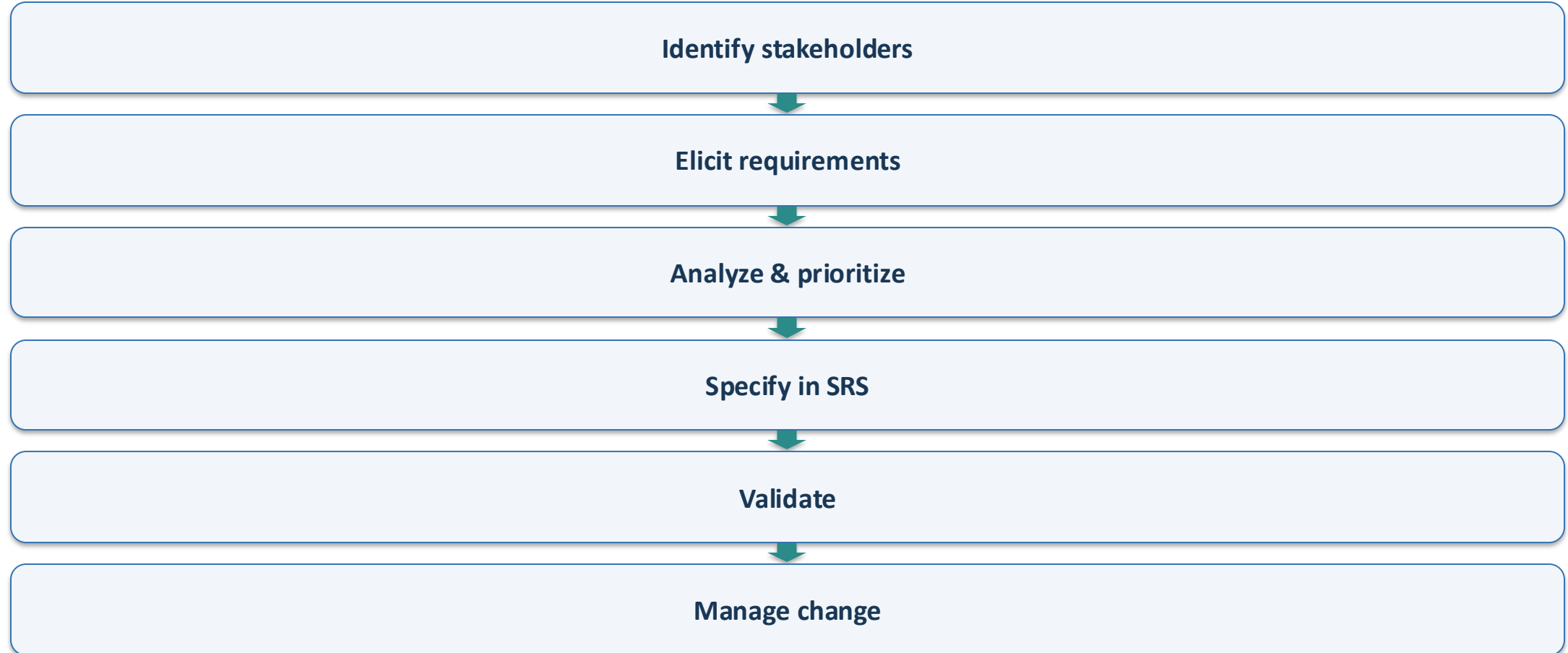
Actor

Condition

Constraint

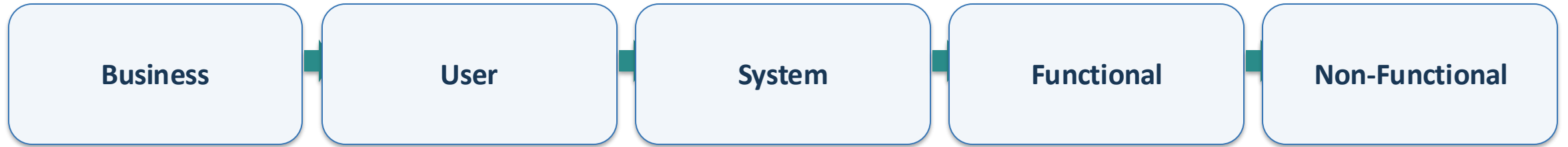
Requirements Engineering Lifecycle

From stakeholder needs to controlled requirements



Types of Requirements

Separate business goals from user needs and system behavior



Business: why the organization needs the system.

User: what a user should accomplish.

System: detailed behavior and constraints.

Functional: services the system performs.

Non-functional: quality attributes and constraints.

Functional Requirements

What the system should do

Examples • E-Commerce

- Create account
- Search product
- Add to cart
- Place order
- Process payment
- Generate invoice

Characteristics

- Services
- Inputs / outputs
- Business rules
- Events & responses
- User goals
- Testable behavior

UML Connection

Functional requirements map naturally to:

Use Cases

→ Sequence diagrams

→ Activity diagrams

Non-Functional Requirements

How well the system must work

Performance

95% requests < 2 sec

Security

Lock after 5 failures

Availability

99.9% monthly

Scalability

100,000 users

Usability

Registration < 3 min

Maintainability

Add provider with low change

Functional vs Non-Functional

Both are essential; they answer different questions

Functional • WHAT

System shall allow users to upload a document.

Focus:

- Feature / service
- Inputs and outputs
- Business behavior
- User goal

Non-Functional • HOW WELL

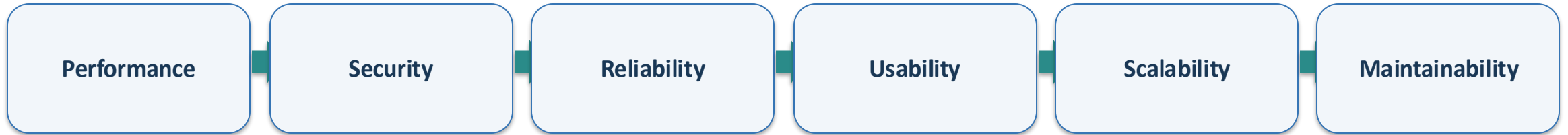
System shall process a 10 MB document within 5 seconds.

Focus:

- Performance
- Security
- Reliability
- Scalability

Non-Functional Requirement Categories

Quality attributes become useful when measurable

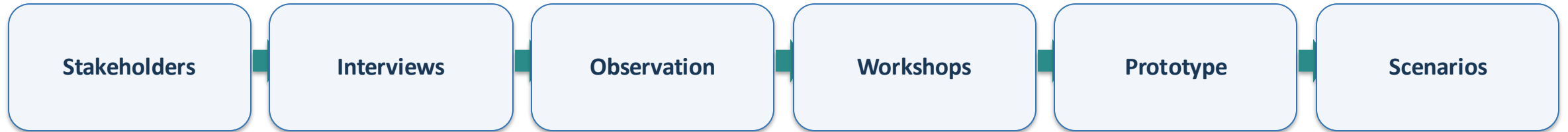


Avoid vague statements such as “the system should be fast.”

Prefer measurable thresholds, operating conditions, and acceptance criteria.

Requirements Elicitation

Requirements are discovered from people, processes and evidence



Goal: uncover explicit needs, hidden workflows, constraints, conflicts and assumptions.

No single technique is sufficient for complex systems.

Elicitation Techniques – When to Use What?

Choose techniques based on stakeholder group and information needed

Technique	Best for	Strength	Typical risk
Interview	Detailed stakeholder knowledge	Deep follow-up	Bias / time
Questionnaire	Large user groups	Scale & metrics	Shallow answers
Observation	Real workflows	Hidden requirements	Observer effect
Workshop	Conflicts & alignment	Fast consensus	Dominant voices
Prototype	UI / unclear needs	Concrete feedback	Users focus on look
Document analysis	Existing systems	Historical evidence	Outdated documents
Scenario	User interactions	Clear behavior	Missed exceptions

Technique 1 – Interviews

Structured, semi-structured and unstructured conversations

Ask

What information do you need before prescribing medication?

What exceptions occur?

What decisions are difficult today?

Capture

- Goals
- Inputs
- Decisions
- Exceptions
- Business rules
- Pain points

Validate

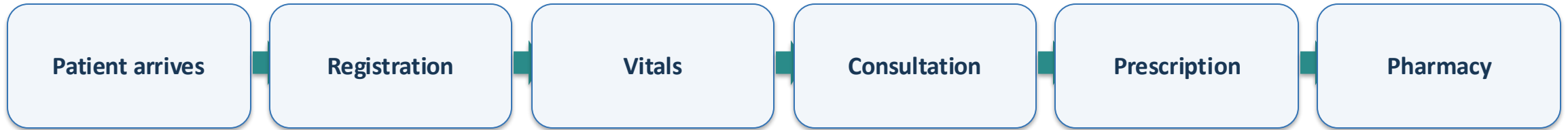
Summarize findings and ask:

“Is this how the process actually works?”

Confirm assumptions with the stakeholder.

Technique 2 – Observation

Observe the real workflow, not only the reported workflow



Observation often exposes implicit requirements, workarounds, exceptions and dependencies that users forget to mention.

Useful for operational systems such as hospitals, manufacturing, logistics and banking.

Workshops, Brainstorming & Prototyping

Collaborative techniques for ambiguity and conflict

Workshop

Multiple stakeholders discuss workflows, conflicts, priorities and acceptance criteria.

Best when different roles have competing needs.

Brainstorming

Generate many ideas quickly.

Example: smart campus → attendance, navigation, parking, food, alerts, security.

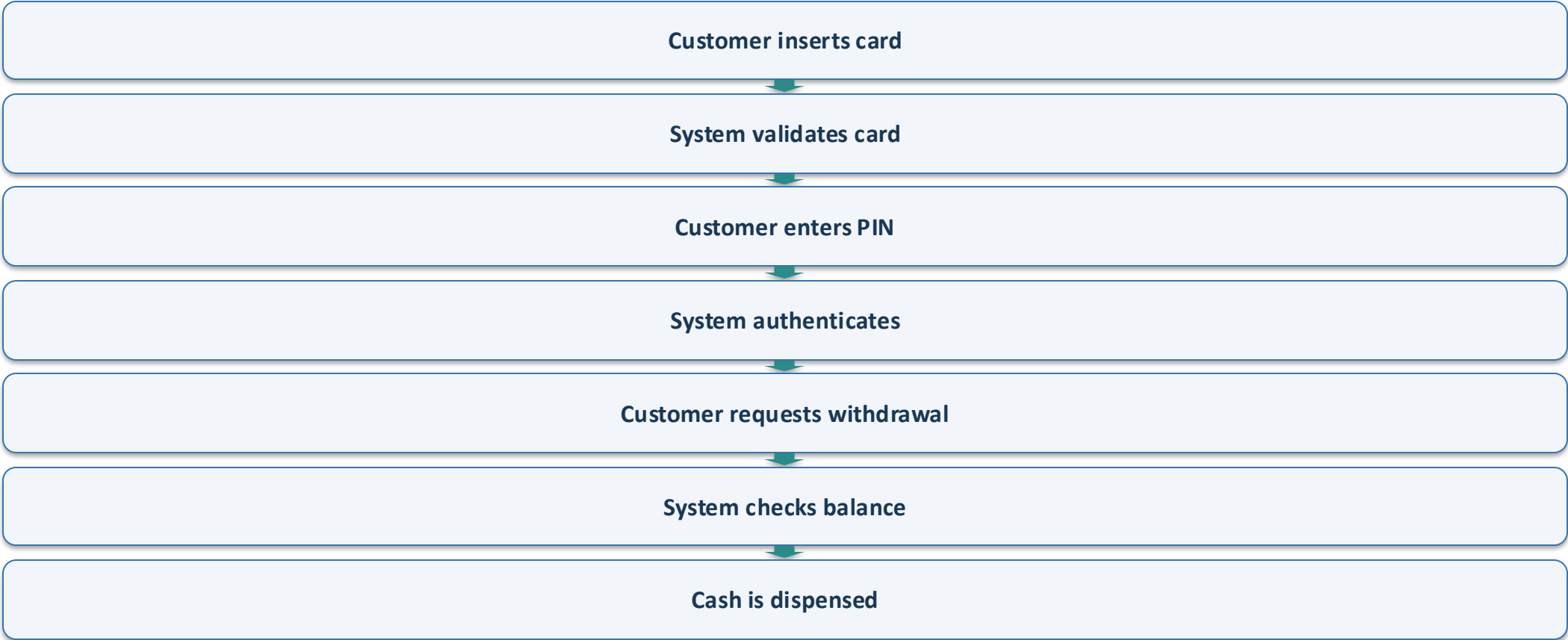
Prototype

Requirement → Prototype → Feedback → Refine.

Especially useful for UI-heavy or uncertain requirements.

Scenario-Based Elicitation

Turn a user goal into observable system behavior



Requirements for AI-Enabled Features

AI adds uncertainty, data and governance requirements

Data

- What data is required?
- Who owns it?
- Is it representative?
- How is quality monitored?
- How is sensitive data protected?

Model

- Required accuracy?
- What if confidence is low?
- How is drift detected?
- What metrics are monitored?
- How is model versioned?

Human & Risk

- Who can override AI?
- What explanations are needed?
- What decisions require approval?
- What is the fallback behavior?

Requirements Analysis & Prioritization

Elicited requirements must be made coherent and actionable



MoSCoW

Must Have • Should Have • Could Have • Won't Have — use this to control release scope.

Requirements Validation

A requirement is valuable only if it can be understood and verified

Correct

Check before baseline

Complete

Check before baseline

Consistent

Check before baseline

Unambiguous

Check before baseline

Feasible

Check before baseline

Verifiable

Check before baseline

Traceable

Check before baseline

Prioritized

Check before baseline

Bad vs Good Requirement

Make quality visible to students

✗ BAD

“The system should be fast.”

Problems:

- Ambiguous
- No threshold
- No load condition
- Not objectively testable

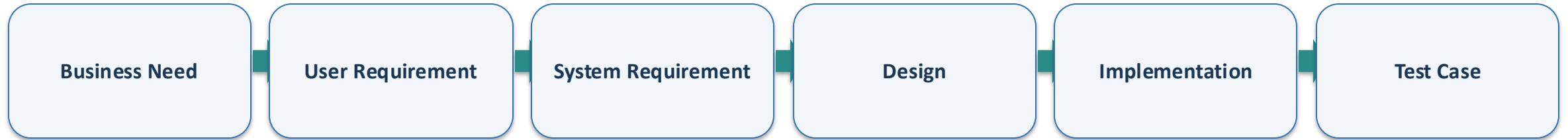
✓ GOOD

“The system shall display the dashboard within 2 seconds for at least 95% of requests under 5,000 concurrent users.”

Specific • measurable • testable

Requirements Traceability

Connect business intent to implementation and testing



Example

REQ-LOGIN-01 → Login Use Case → Authentication Module → Login Test Cases

Software Requirements Specification (SRS)

The controlled contract for what the software is expected to provide

Purpose

- Customer–developer agreement
- Basis for design
- Basis for testing
- Supports estimation
- Supports maintenance
- Controls future enhancement

Good SRS Properties

- Clear
- Complete
- Consistent
- Verifiable
- Traceable
- Modifiable
- Prioritized
- Version controlled

Typical SRS Structure

A practical structure for student project documentation



Use Case Specification within SRS

Move beyond a one-line use case name

Book Appointment

Actor: Patient

Precondition: authenticated

Goal: reserve an available slot

Postcondition: appointment stored

Main Flow

1. Select doctor
2. Display slots
3. Select slot
4. Validate availability
5. Confirm appointment

Each step should be observable.

Alternative Flow

Selected slot becomes unavailable → system asks the patient to choose another slot.

Also document exceptions, business rules and acceptance criteria.

Why Do Requirements Change?

Change is normal; uncontrolled change is the problem

Business

Strategy, market, competition, customer expectations.

Technology

New platforms, APIs, cloud services, security threats.

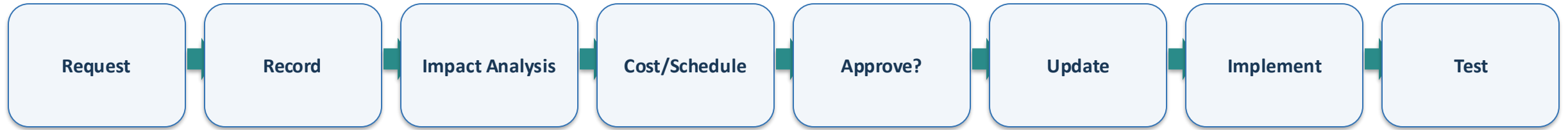
External

Regulations, policies, standards, legal/privacy constraints.

Key idea: every change can propagate into design, code, tests, schedule and cost.

Requirements Change Management

A change request must be evaluated before it enters the baseline

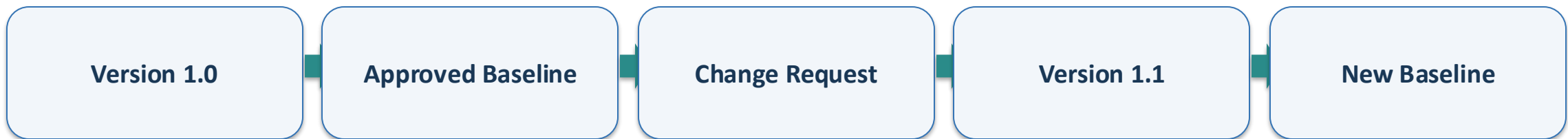


Impact Analysis

Technical • Cost • Schedule • Testing • Security • Data •
Dependencies

Baseline and Requirements Volatility

Control versions and identify what is likely to change



High Volatility

UI, reports, business rules, customer preferences

Lower Volatility

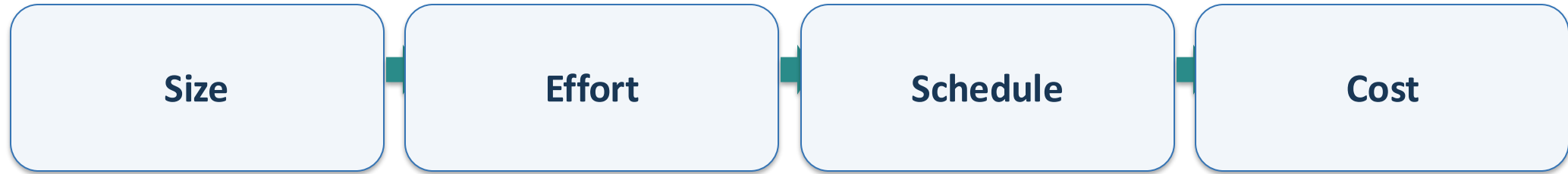
Core objectives, legal constraints, fundamental security policies

OBJECT-ORIENTED SOFTWARE ESTIMATION

From requirements and models to effort, schedule and cost

Why Software Estimation?

Management needs a defensible forecast before committing resources



Estimate early to support feasibility, staffing, budgeting, release planning and risk management.

Do not confuse an estimate with a promise: uncertainty should be made explicit.

Why OO Software Estimation is Difficult

Object orientation changes the units and uncertainty of development

Reuse

Frameworks, libraries, components and inheritance can reduce or reshape implementation effort.

Behavior

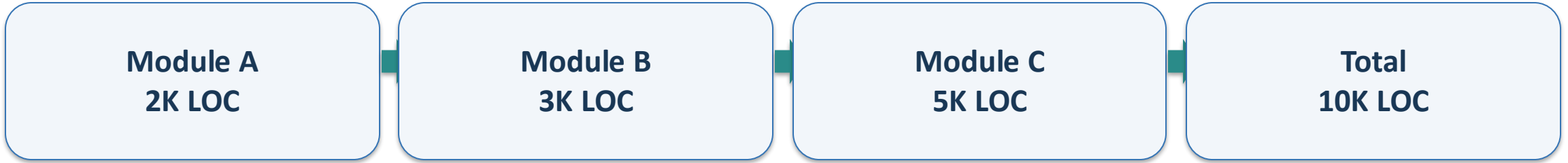
Dynamic interactions, patterns and distributed components make behavior harder to size early.

Uncertainty

Changing requirements, integrations and third-party dependencies affect estimates.

Lines of Code (LOC)

Simple concept, but often weak for early OO estimation

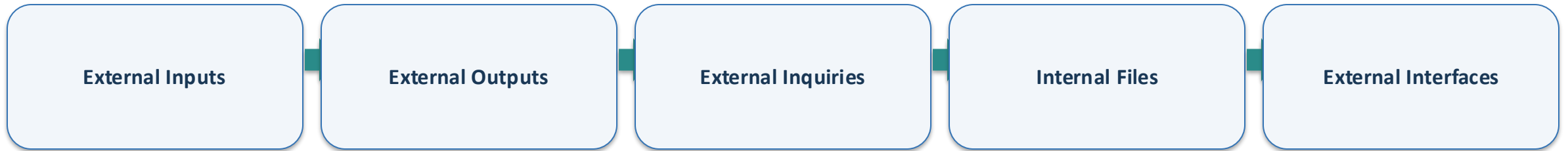


Advantage: easy to understand after implementation.

Limitations: difficult to estimate early; language-dependent; encourages premature design assumptions.

Function Point Analysis

Measure delivered functionality rather than source-code volume



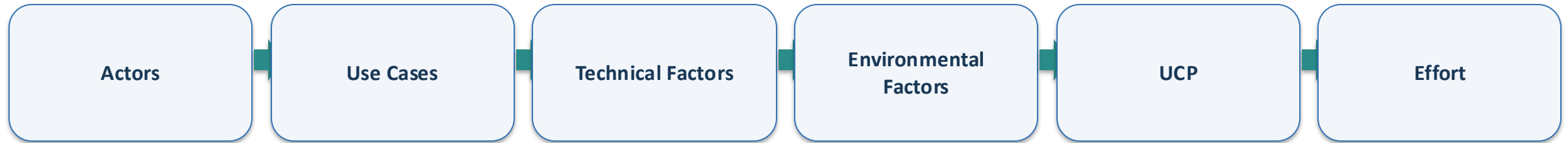
Useful when source code is unknown.

Focuses on user-visible functionality and data interactions.

Can be converted to effort using historical productivity.

Use Case Points (UCP)

A natural early-estimation approach for use-case-driven OO projects



Why UCP?

Use cases are often available before detailed classes and source code, so UCP supports early estimation directly from the OO requirements model.

Use Case & Actor Complexity

Complexity is driven by interaction and transaction count

Simple

Use Case: few transactions

Example: View Profile

Actor: defined API

Average

Use Case: moderate interaction

Example: Book Appointment

Actor: external system/protocol

Complex

Use Case: many interactions,
alternatives, external systems

Example: Process Insurance Claim

Actor: human GUI

Use Case Points – Formula

Core relationship used in a commonly taught UCP approach

$$\text{UCP} = (\text{UAW} + \text{UUCW}) \times \text{TCF} \times \text{ECF}$$

UAW

Unadjusted Actor Weight

UUCW

Unadjusted Use Case Weight

TCF

Technical Complexity Factor

ECF

Environmental Complexity Factor

$$\text{Effort} = \text{UCP} \times \text{Productivity Factor}$$

UCP – Worked Example

Show students how an abstract model becomes a quantitative estimate

Given

$$UAW = 12$$

$$UUCW = 80$$

$$TCF = 0.9$$

$$ECF = 0.8$$

$$UCP = (12 + 80) \times 0.9 \times 0.8$$

$$UCP = 73.44$$

Effort

$$\text{Productivity} = 20 \text{ person-hours / UCP}$$

$$\text{Effort} = 73.44 \times 20$$

$$= 1,468.8 \text{ person-hours}$$

Convert to person-months using the organization's working-hour convention.

COCOMO / COCOMO II

Algorithmic estimation models complement OO-specific techniques

COCOMO

$$\text{Effort} = a(\text{KLOC})^b$$

KLOC = thousand lines of code.

Parameters depend on project mode/model calibration.

Useful when size can be estimated.

COCOMO II

Better suited to modern development contexts.

Considers scale, reuse, team capability, platform complexity, process and reliability.

OO projects should account for reuse and frameworks.

Other Estimation Techniques

Triangulate estimates rather than trusting one number

Analogy

Compare with historical projects.

Adjust for scope, technology, integrations, security and team capability.

Expert Judgment

Use experienced developers / managers.

Combine independent estimates to reduce individual bias.

Three-Point / PERT

O = optimistic

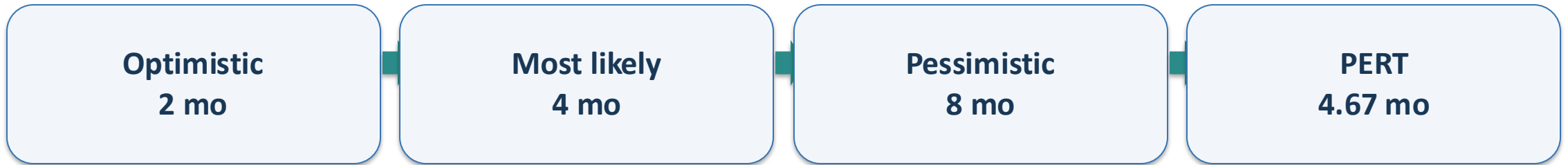
M = most likely

P = pessimistic

$$E = (O + 4M + P) / 6$$

Three-Point Estimation – Example

Explicitly represent uncertainty in schedule estimates

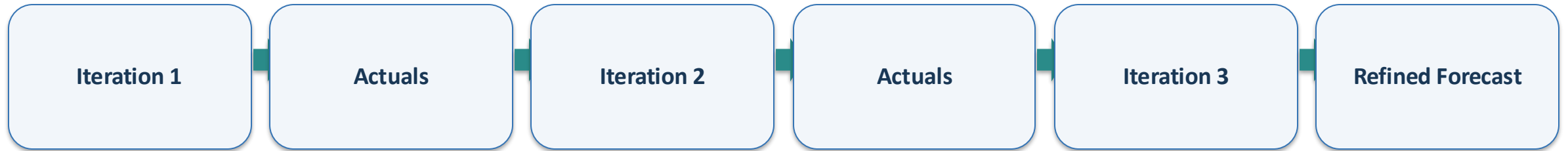


$$E = [2 + (4 \times 4) + 8] / 6 = 4.67 \text{ months.}$$

The range communicates risk better than a single optimistic estimate.

Estimation in Iterative / Agile OO Development

Estimates should become better as evidence accumulates

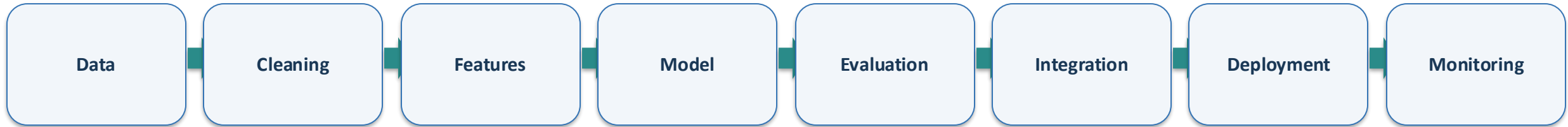


Use completed functionality, velocity/productivity, defect trends and changed requirements to update forecasts.

Re-estimation is not failure; it is evidence-based project control.

AI/ML Projects – Additional Estimation Factors

AI introduces data and model uncertainty beyond conventional software

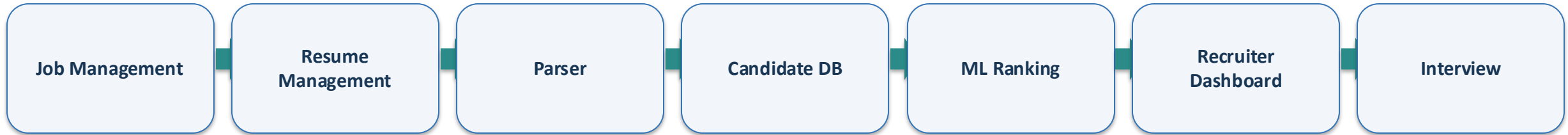


Estimation Insight

A well-defined software requirement can still contain high AI uncertainty because dataset quality, model performance, experimentation and drift are not fully known upfront.

Case Study – AI-Assisted Recruitment

AI is a component inside a broader software system

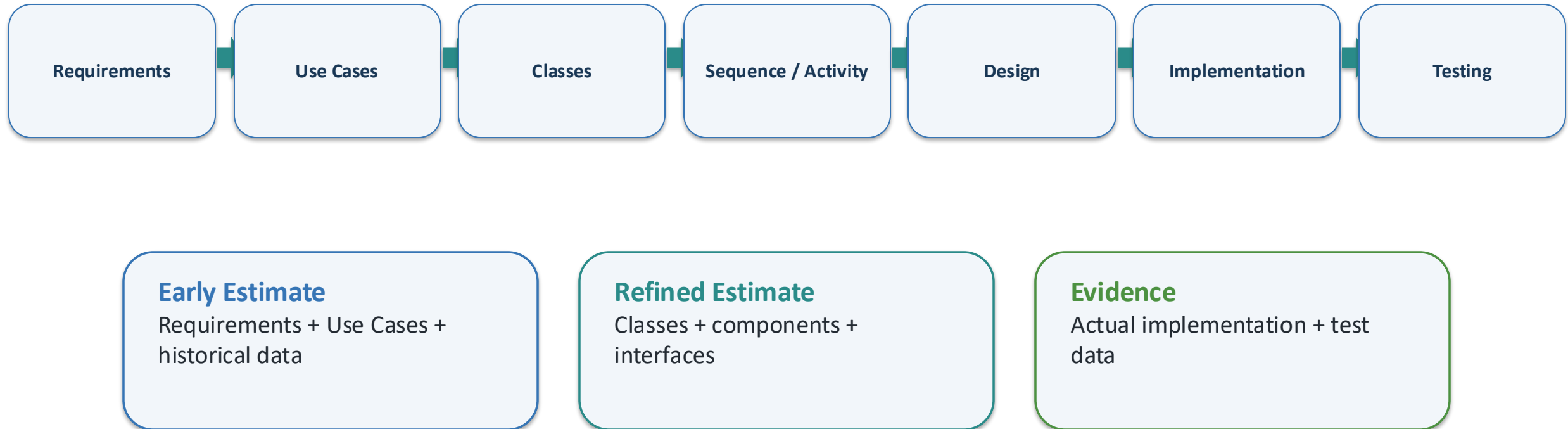


Estimate conventional modules separately from data preparation, model experimentation, validation, integration, monitoring and governance.

Keep final recruitment decisions under authorized human control.

Requirements → UML → Estimation

A coherent OO workflow



Common Estimation Mistakes

What students should learn to challenge in project proposals

Overconfidence

Ask: what evidence supports this estimate?

Ignoring NFRs

Ask: what evidence supports this estimate?

Ignoring integration

Ask: what evidence supports this estimate?

Ignoring testing

Ask: what evidence supports this estimate?

Ignoring rework

Ask: what evidence supports this estimate?

Ignoring technical debt

Ask: what evidence supports this estimate?

Master's-Level Case Study – Smart University Management

Use one system to connect RE, UML and estimation

Scope

Registration
Course enrollment
Attendance
Examination
Results
Faculty management
Performance analysis

Stakeholders

Student
Faculty
Administrator
Examination department
HOD
Management

AI Support

Predict students who may require academic intervention.

Do not treat AI as the whole system: model, data, UI, workflow, security and governance are all software concerns.

Classroom Exercise – Requirement Quality

Ask students to critique, rewrite and test requirements

Requirement A

“The system should be user-friendly.”

Discuss:

- What is ambiguous?
- How would you measure it?
- What acceptance test could prove it?

Requirement B

“The system shall display the dashboard within 2 seconds for at least 95% of requests under 5,000 concurrent users.”

Discuss why it is specific, measurable and testable.

Project Assignment – Requirements + OO Estimation

Expected deliverables for master's students

1 • Requirements

Stakeholders
Business requirements
15 functional requirements
10 non-functional requirements
Prioritization

2 • Elicitation + SRS

4 elicitation techniques
SRS introduction & scope
FR / NFR
Constraints
Assumptions
Use cases

3 • UML

Use Case
Class
Sequence
Activity
State
Component
Deployment
Package

4 • Estimation

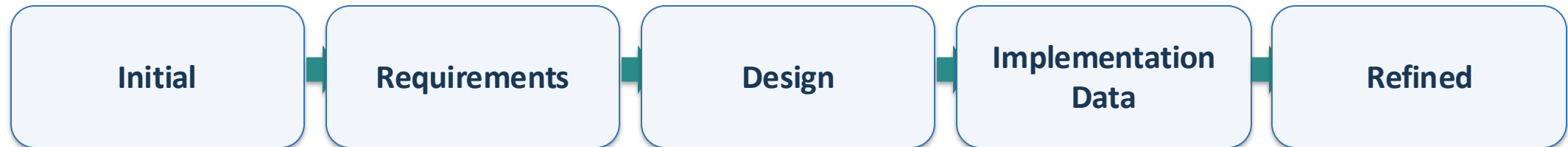
Expert
Three-point
Use Case Points
COCOMO/COCOMO II discussion
Risks

Key Conceptual Question

Can we estimate accurately before requirements are finalized?

Short answer

Not with complete accuracy. We can create progressively better estimates as uncertainty decreases.



Final Concept Map

The module as one engineering story

